

MODELO DE DISEÑO ORIENTADO A ATRIBUTOS (ADD)

Sistema de Gestión y Monitoreo de Salud de Infraestructura Vial (SmartRoad)

Por:

Sebastián Osorio Tuberquia
Alejandro De Mendoza
Federico López Montoya
Deiby Olaya Orejuela

Institución Universitaria Politécnico Grancolombiano
Maestría en Arquitectura de Software
Diseño Arquitectónico de Software
Actividad Sumativa — Unidad 4
Profesor: John Rondón Suárez
Junio 2026

1. Propósito del Informe

El presente informe documenta la aplicación del método de Diseño Orientado a Atributos (ADD, por sus siglas en inglés Attribute-Driven Design) para la construcción de un diseño arquitectónico preliminar del sistema SmartRoad, una plataforma de software destinada a evaluar, monitorear y gestionar el deterioro de la infraestructura vial en contextos urbanos y rurales.

La metodología ADD, desarrollada en el Software Engineering Institute (SEI) de la Universidad Carnegie Mellon, provee un proceso iterativo y estructurado que parte de los atributos de calidad del sistema —también denominados drivers arquitectónicos— para guiar la selección de conceptos y estructuras de diseño. A diferencia de enfoques funcionales, ADD prioriza la satisfacción de requisitos no funcionales críticos desde las etapas iniciales del proceso, lo que reduce el riesgo técnico y facilita la toma de decisiones con fundamento arquitectónico sólido.

En el presente documento se presentan dos iteraciones completas del método ADD: la primera orientada a establecer la arquitectura de referencia y la vista de contexto del sistema, y la segunda enfocada en la descomposición modular interna, la selección de patrones de diseño y la definición de la vista de contenedores. Cada iteración documenta los pasos del proceso, los conceptos de diseño evaluados, las alternativas descartadas, las responsabilidades asignadas a los elementos arquitectónicos y la revisión del grado de satisfacción de los drivers.

El propósito final es proporcionar un artefacto técnico que sirva como base para la implementación del sistema SmartRoad, garantizando que las decisiones de diseño adoptadas sean coherentes con los objetivos de calidad, las restricciones del entorno y los aspectos externos identificados en el proceso de captura de insumos.

2. Descripción del Contexto o Situación

Los sistemas de infraestructura vial representan uno de los activos más críticos para el desarrollo económico y social de cualquier país. En Colombia, como en gran parte de América Latina, el deterioro prematuro de vías primarias, secundarias y terciarias genera pérdidas millonarias en transporte, accidentalidad y costo de mantenimiento reactivo. La ausencia de sistemas de monitoreo continuo obliga a las entidades gubernamentales y concesionarias a depender de inspecciones visuales periódicas, altamente costosas, subjetivas y con baja cobertura territorial.

SmartRoad es una plataforma de software que responde a esta problemática mediante la integración de tecnologías de sensorización IoT, procesamiento de imágenes con inteligencia artificial, análisis de datos geoespaciales y modelos predictivos de deterioro vial. El sistema permite a ingenieros viales, entidades gubernamentales y concesionarios de vías obtener información actualizada sobre el estado estructural de la malla vial, identificar zonas críticas de deterioro, priorizar inversiones de mantenimiento y generar reportes de cumplimiento regulatorio.

El siguiente prompt fue usado con un modelo de inteligencia artificial generativa para obtener los insumos de diseño iniciales del sistema:

“Generar un listado simulado compuesto por cinco elementos para cada aspecto de los insumos de diseño: objetivos de diseño, casos de uso y actores relacionados, escenarios de atributos de calidad, atributos externos y restricciones. Este enfoque se centra en un *software* destinado a evaluar el deterioro de la malla vial, aportando información clave sobre su estado y funcionalidad en diferentes contextos urbanos y rurales. Además, amplíe la descripción del enfoque.”

A partir de los insumos generados, se complementaron y refinaron los elementos con base en el conocimiento técnico del equipo, garantizando la coherencia entre los requisitos del dominio vial y los principios del método ADD.

3. Insumos o Entradas de Diseño

Los siguientes insumos de diseño fueron generados mediante el uso del modelo de inteligencia artificial y posteriormente refinados con criterio técnico por el equipo. Constituyen la base sobre la cual se aplicaron las iteraciones del método ADD.

3.1 Casos de Uso

ID	Caso de Uso	Descripción
UC-1	Registro y autenticación de usuarios	Ingenieros viales, supervisores de concesión y funcionarios gubernamentales crean perfiles institucionales y acceden al sistema mediante autenticación multifactor.
UC-2	Captura y carga de datos de sensores IoT	El sistema recibe, valida e ingiere en tiempo real datos provenientes de sensores de aceleración, temperatura, humedad y vibración instalados en segmentos viales monitoreados.
UC-3	Análisis de imágenes para detección de deterioro	Permite cargar imágenes o videos de la superficie vial para que el motor de IA detecte automáticamente fisuras, baches, hundimientos y otros tipos de daño estructural.
UC-4	Generación de reportes y alertas de priorización	El sistema genera reportes de estado estructural por segmento vial, alertas automáticas de deterioro crítico y recomendaciones de intervención según modelos predictivos de ciclo de vida.
UC-5	Visualización geoespacial del estado de la malla vial	Los usuarios pueden consultar un mapa interactivo que muestra el índice de condición del pavimento (ICP) por segmento, capas de calor de zonas críticas y evolución histórica del deterioro.

3.2 Escenarios de Atributos de Calidad

ID	Atributo de Calidad	Escenario
QA-1	Disponibilidad	El sistema debe estar disponible el 99.9 % del tiempo (≤ 8.7 horas de inactividad anual) para garantizar la recepción continua de datos de sensores y el acceso a reportes por parte de entidades gubernamentales.
QA-2	Rendimiento	El procesamiento de imágenes para detección de deterioro debe completarse en un tiempo máximo de 10 segundos por imagen bajo carga normal, y no superar 30 segundos en condiciones de alta concurrencia.
QA-3	Escalabilidad	La plataforma debe soportar la incorporación de hasta 10.000 nuevos sensores IoT sin degradación del rendimiento del pipeline de ingesta de datos, mediante escalado horizontal automático.

QA-4	Seguridad	Toda transmisión de datos de sensores y toda comunicación entre componentes deben estar protegidas mediante cifrado en tránsito (TLS 1.3) y en reposo (AES-256). El acceso a funcionalidades críticas requiere autenticación multifactor y autorización basada en roles (RBAC).
QA-5	Modificabilidad	La incorporación de nuevos modelos de IA para detección de tipos adicionales de deterioro debe poder realizarse sin afectar los módulos de ingesta de datos ni la capa de visualización, requiriendo únicamente cambios en el módulo de inferencia.

3.3 Restricciones

ID	Restricción	Descripción
R-1	Regulación de datos geoespaciales	El sistema debe cumplir con la política de datos abiertos del IGAC (Instituto Geográfico Agustín Codazzi) y la normativa CONPES 3700 sobre infraestructura de datos espaciales en Colombia.
R-2	Interoperabilidad con sistemas gubernamentales	El sistema debe ser capaz de integrarse con el Sistema de Información de Carreteras (SINC) del INVIAS mediante APIs REST estándar o intercambio de datos en formato GeoJSON.
R-3	Presupuesto y tecnología disponible	El diseño debe priorizar el uso de servicios cloud en modalidad pay-as-you-go para minimizar la inversión inicial en infraestructura. La plataforma cloud de referencia es AWS.
R-4	Conectividad intermitente en zonas rurales	Los nodos de sensores instalados en vías rurales operan en zonas con conectividad limitada o intermitente, por lo que el sistema debe soportar modos de operación offline con sincronización diferida.
R-5	Estándares de interoperabilidad de datos viales	Los modelos de deterioro generados deben ser exportables en formato compatible con el estándar PASER (Pavement Surface Evaluation and Rating) y el protocolo LTPP (Long-Term Pavement Performance).

3.4 Aspectos Externos

ID	Aspecto Externo	Descripción
AE-1	Ecosistema de IA para visión computacional	La evolución acelerada de modelos de visión computacional (YOLO, SAM, Detectron2) representa una oportunidad para mejorar continuamente la precisión del sistema en detección de deterioro.
AE-2	Políticas de contratación pública de tecnología	Los ciclos de licitación de proyectos viales en Colombia imponen ventanas de tiempo específicas para la integración del sistema con proyectos de concesión 4G y 5G.
AE-3	Comportamiento y experiencia del usuario técnico	Los ingenieros viales usuarios del sistema tienen alta experiencia en herramientas GIS (ArcGIS, QGIS) y esperan interfaces de visualización de calidad comparable.

AE-4	Evolución de estándares IoT para infraestructura	Estándares emergentes como ETSI ITS y ISO 21217 para comunicación vehículo-infraestructura pueden requerir adaptaciones del módulo de ingesta de datos en el mediano plazo.
AE-5	Variabilidad climática y condiciones operativas	Las condiciones de lluvia intensa, neblina y contaminación visual en zonas rurales afectan la calidad de las imágenes captadas, requiriendo preprocesamiento adaptativo de datos.

3.5 Insumos Seleccionados para el Proceso de Diseño ADD

La siguiente tabla sintetiza los insumos que serán considerados en las iteraciones de diseño, priorizados en función de la importancia para el cliente y la dificultad de implementación estimada por el arquitecto:

Categoría	Detalles
Propósito del diseño	Este es un sistema de software nuevo. La organización implementará el desarrollo usando metodología ágil con sprints de dos semanas. Se requiere un diseño arquitectónico inicial que sirva como referencia para las decisiones de implementación y permita evaluar los riesgos técnicos desde las primeras iteraciones.
Requerimientos funcionales primarios	UC-2 / UC-3 / UC-5
Escenarios de atributos de calidad	QA-1 (Disponibilidad) — Alta importancia / Media dificultad QA-2 (Rendimiento) — Alta importancia / Alta dificultad QA-3 (Escalabilidad) — Alta importancia / Alta dificultad QA-4 (Seguridad) — Alta importancia / Media dificultad QA-5 (Modificabilidad) — Media importancia / Media dificultad
Restricciones	R-1 / R-2 / R-3 / R-4 / R-5
Aspectos externos	AE-1 / AE-3 / AE-4

4. Desarrollo del Método ADD

A continuación se presentan las dos iteraciones del método ADD aplicadas al sistema SmartRoad. Cada iteración sigue los pasos definidos por la metodología: revisión de insumos, establecimiento del objetivo, selección del elemento a refinar, elección del concepto de diseño, instanciación de elementos, registro de decisiones y revisión de alcance.

4.1 Iteración 1: Arquitectura de Referencia y Vista de Contexto

Paso 1 — Revisión de los Insumos de Diseño

Antes de iniciar la primera iteración se verificó la completitud y consistencia de los insumos capturados. Se confirmó la disponibilidad de los cinco casos de uso, los cinco escenarios de atributos de calidad (QA-1 a QA-5), las cinco restricciones (R-1 a R-5) y los cinco aspectos externos (AE-1 a AE-5). Dado que SmartRoad es un sistema nuevo, no existe arquitectura existente que deba considerarse. Los insumos fueron revisados sin modificaciones, dado que la priorización realizada en la sección 3.5 ya establece un orden de atención para las iteraciones.

Paso 2 — Establecer el Objetivo de la Iteración y Seleccionar los Drivers

El objetivo de la primera iteración es definir la arquitectura de referencia del sistema SmartRoad y construir la vista de contexto (nivel 1 del modelo C4). Se busca establecer las fronteras del sistema, los actores externos y los sistemas con los que interactúa, así como tomar la decisión estructural más importante: el estilo arquitectónico predominante.

Drivers seleccionados para esta iteración:

Funcionales: UC-2, UC-3, UC-5 (los tres casos de uso de mayor peso arquitectónico)

Calidad: QA-1 (Disponibilidad), QA-2 (Rendimiento), QA-3 (Escalabilidad)

Restricciones: R-3 (Cloud AWS), R-4 (Offline IoT)

Aspectos externos: AE-1 (IA/CV), AE-4 (Estándares IoT)

Paso 3 — Elegir los Elementos del Sistema para Refinar

Al ser la primera iteración de un sistema nuevo, el elemento a refinar es el sistema en su totalidad considerado como una caja negra. Se busca determinar su posición en el ecosistema externo, identificar todos los actores y sistemas con los que interactúa, y establecer el estilo arquitectónico que guiará las decisiones de las iteraciones subsiguientes. Este nivel de abstracción corresponde al Diagrama de Contexto del modelo C4.

Paso 4 — Elegir Conceptos de Diseño que Satisfagan los Drivers

Se evaluaron tres alternativas de concepto de diseño. A continuación se presenta el concepto seleccionado y, posteriormente, las alternativas descartadas con su justificación:

Concepto seleccionado:

Concepto de Diseño	Justificación
Arquitectura de referencia orientada a eventos (EDA) sobre base de microservicios	La combinación de Event-Driven Architecture (EDA) con microservicios resuelve directamente los drivers más críticos del sistema: • QA-1 (Disponibilidad): la descomposición en servicios independientes permite que el fallo de un componente no afecte la disponibilidad global del sistema. • QA-2 (Rendimiento): el procesamiento asíncrono de eventos desacopla la ingesta de datos del procesamiento de IA, evitando cuellos de botella. • QA-3 (Escalabilidad): cada microservicio puede escalar horizontalmente de forma independiente según la demanda. • QA-5 (Modificabilidad): la incorporación de nuevos modelos de IA se realiza como un nuevo consumidor de eventos sin modificar otros componentes. • R-4: el diseño de agentes IoT con buffer local soporta la operación offline con sincronización diferida.

Alternativas descartadas:

Alternativa	Justificación para Descartarla
Arquitectura monolítica en capas (N-Tier)	Si bien una arquitectura monolítica simplifica el desarrollo inicial, su escalabilidad vertical tiene límites físicos que comprometen QA-3. El acoplamiento entre los módulos de ingesta de sensores, procesamiento de IA y visualización geoespacial genera dependencias que dificultan la incorporación de nuevos modelos (QA-5) y crean un único punto de fallo que afecta la disponibilidad (QA-1). Adicionalmente, los ciclos de despliegue de todo el sistema para modificaciones parciales aumentan el riesgo operativo.
Arquitectura serverless pura (FaaS)	El modelo serverless puro presenta limitaciones críticas para este sistema: los tiempos de arranque en frío (cold starts) son incompatibles con el requisito de procesamiento de imágenes en ≤ 10 segundos (QA-2) cuando los modelos de IA tienen tamaños considerables. Asimismo, la naturaleza stateless de las funciones dificulta la gestión de streams continuos de datos IoT (UC-2) y el mantenimiento de conexiones persistentes con los sensores en campo.

Paso 5 — Instanciar Elementos, Establecer Responsabilidades y Definir Interfaces

A partir del concepto de diseño seleccionado se instancian los elementos arquitectónicos que conforman el Diagrama de Contexto del sistema SmartRoad. La siguiente tabla detalla cada elemento y su responsabilidad:

Elemento	Responsabilidad
Ingeniero Vial / Supervisor de Concesión	Actor humano principal. Accede a la plataforma para consultar el estado de la malla vial, generar reportes y gestionar alertas de deterioro crítico. Interactúa a través del portal web.

Entidad Gubernamental (INVIAS / Gobernación)	Actor institucional que consume reportes consolidados y datos de cumplimiento regulatorio. Requiere exportación de datos en formatos estándar (GeoJSON, PASER).
Sistema SmartRoad	Plataforma central del sistema. Orquesta la ingesta de datos de sensores IoT, el procesamiento de imágenes con IA, el almacenamiento geoespacial y la generación de reportes y alertas. Expone APIs REST y WebSocket a los actores externos e internos.
Red de Sensores IoT Viales	Sistema externo compuesto por nodos de sensorización instalados en la infraestructura vial. Transmite datos de aceleración, temperatura, vibración y humedad al sistema via MQTT/HTTPS con soporte offline-first.
Sistema SINC — INVIAS	Sistema externo del Instituto Nacional de Vías. Recibe datos de estado de vías desde SmartRoad mediante API REST en formato GeoJSON para integración con el inventario nacional de carreteras.
Proveedor de Imágenes Satelitales / Drones	Sistema externo que suministra imágenes aéreas y satelitales de alta resolución para el análisis de deterioro superficial extensivo en segmentos de alta prioridad.

Paso 6 — Esbozar Vistas y Registrar Decisiones de Diseño

La vista generada en esta iteración es el Diagrama de Contexto (C4 - Nivel 1) del sistema SmartRoad. A continuación se describe textualmente la estructura representada en el diagrama:

El sistema SmartRoad se sitúa en el centro del ecosistema. El Ingeniero Vial y el Supervisor de Concesión interactúan directamente con el sistema a través del portal web para registrar datos de campo, consultar reportes y gestionar alertas. La Entidad Gubernamental (INVIAS / Gobernación) recibe reportes consolidados y datos de cumplimiento a través de una interfaz de exportación estándar. La Red de Sensores IoT Viales transmite datos de aceleración, temperatura y vibración al sistema mediante el protocolo MQTT con soporte offline-first. El Sistema SINC del INVIAS recibe actualizaciones de estado vial a través de una API REST en formato GeoJSON. El Proveedor de Imágenes Satelitales/Drones suministra imágenes de alta resolución para el motor de análisis IA del sistema.

Decisiones de diseño registradas:

DD-1.1: Se adopta la arquitectura orientada a eventos (EDA) sobre base de microservicios como estilo arquitectónico de referencia del sistema SmartRoad, por su capacidad de satisfacer QA-1, QA-2, QA-3 y QA-5 de forma simultánea.

DD-1.2: Se establece MQTT como protocolo de comunicación entre los sensores IoT y el sistema, con soporte para operación offline-first mediante buffer local en los nodos de campo (R-4).

DD-1.3: La integración con el sistema SINC del INVIAS se realizará mediante API REST con intercambio de datos en formato GeoJSON, cumpliendo R-2.

DD-1.4: El despliegue del sistema se realizará sobre infraestructura AWS en modalidad cloud-native (R-3), priorizando servicios administrados para reducir la carga operativa.

Paso 7 — Revisar el Objetivo de la Iteración y los Alcances del Diseño

La siguiente tabla muestra el estado de satisfacción de los insumos de diseño al finalizar la Iteración 1:

Insumo de diseño	Estado	Elemento del concepto de diseño
UC-2	Parcial	Componente de ingesta IoT del Sistema SmartRoad — relación con Red de Sensores
UC-3	Parcial	Motor de IA del Sistema SmartRoad — relación con Proveedor de Imágenes
UC-5	Parcial	Módulo de visualización del Sistema SmartRoad — interfaz con usuarios
QA-1	Parcial	Arquitectura distribuida — sin single point of failure en el contexto del sistema
QA-2	Pendiente	Aún sin decisiones — requiere descomposición interna (Iteración 2)
QA-3	Pendiente	Aún sin decisiones — requiere descomposición interna (Iteración 2)
QA-4	Parcial	Identificados límites de sistema con actores externos — autenticación pendiente de detalle
QA-5	Pendiente	Aún sin decisiones — requiere descomposición modular (Iteración 2)
R-1 / R-2	Pendiente	Identificados actores regulatorios en contexto — detalle de integración pendiente
R-3	Parcial	Arquitectura cloud-native adoptada — elección de servicios AWS pendiente
R-4	Parcial	Modo offline-first reconocido en nodo IoT externo — detalle pendiente
R-5	Pendiente	Aún sin decisiones — formatos de exportación pendientes de diseño
AE-1 / AE-3 / AE-4	Pendiente	Considerados en la selección del concepto — sin elementos arquitectónicos concretos aún

Conclusión de la revisión: los casos de uso primarios (UC-2, UC-3, UC-5) han sido abordados parcialmente a nivel de contexto. Los atributos QA-2, QA-3 y QA-5 permanecen pendientes de detalle, lo que justifica la realización de la Iteración 2 con foco en la descomposición interna del sistema. Las restricciones R-1, R-5 y los aspectos externos requieren refinamiento en iteraciones subsiguientes.

4.2 Iteración 2: Descomposición Interna — Vista de Contenedores

Paso 1 — Revisión de los Insumos de Diseño

Al inicio de la segunda iteración se revisaron los insumos de diseño con base en los resultados de la Iteración 1. Se tomó como punto de partida el diseño de contexto resultante (DD-1.1 a DD-1.4) y se identificaron los insumos pendientes de resolución: QA-2, QA-3, QA-5, R-4 y AE-1. Adicionalmente, se incorporó el diseño de arquitectura existente (resultado de la Iteración 1) como insumo adicional, dado que esta no es la primera ronda de diseño.

Paso 2 — Establecer el Objetivo de la Iteración y Seleccionar los Drivers

El objetivo de la segunda iteración es descomponer el sistema SmartRoad en sus contenedores internos (nivel 2 del modelo C4), definir las tecnologías específicas de implementación y establecer los mecanismos que resuelven los atributos de calidad pendientes de la Iteración 1.

Drivers seleccionados para esta iteración:

Calidad: QA-2 (Rendimiento), QA-3 (Escalabilidad), QA-4 (Seguridad), QA-5 (Modificabilidad)

Restricciones: R-3 (Servicios AWS), R-4 (Offline IoT), R-5 (Formatos de exportación)

Aspectos externos: AE-1 (Modelos de IA), AE-3 (Experiencia GIS), AE-4 (Estándares IoT)

Paso 3 — Elegir los Elementos del Sistema para Refinar

El elemento a refinar en esta iteración es el Sistema SmartRoad tal como fue definido en la Iteración 1. Se procede a su descomposición interna para identificar los contenedores tecnológicos que lo conforman, sus responsabilidades individuales, las tecnologías asignadas y las interfaces de comunicación entre ellos. Este nivel de detalle corresponde al Diagrama de Contenedores del modelo C4.

Paso 4 — Elegir Conceptos de Diseño que Satisfagan los Drivers

Concepto seleccionado:

Concepto de Diseño	Justificación
Descomposición en microservicios funcionales con Event Bus (Apache Kafka) y pipeline de procesamiento IA asíncrono	La descomposición en microservicios especializados con Apache Kafka como bus de eventos central permite: • QA-2 (Rendimiento): el procesamiento de imágenes se realiza de forma asíncrona por el Motor de IA, desacoplando la respuesta al usuario de los tiempos de inferencia del modelo. • QA-3 (Escalabilidad): cada microservicio puede desplegarse en múltiples instancias con EKS (Kubernetes en AWS), con autoescalado basado en métricas de carga. • QA-5 (Modificabilidad): la incorporación de nuevos modelos de IA se realiza desplegando un nuevo consumidor Kafka en el tópico de imágenes sin afectar otros servicios. • QA-1 (Disponibilidad): Kafka garantiza la durabilidad de los mensajes y permite que los servicios procesen eventos en diferido si hay fallas transitorias.

	• R-4: el Servicio de Ingesta IoT implementa buffer local en los agentes de campo con sincronización Kafka cuando se restaura la conectividad.
--	--

Alternativas descartadas:

Alternativa	Justificación para Descartarla
Comunicación síncrona REST entre microservicios	La comunicación REST síncrona entre microservicios crea acoplamiento temporal: si el Motor de IA está bajo alta carga, el Servicio de Ingesta queda bloqueado esperando respuesta, degradando el rendimiento global (QA-2). Adicionalmente, en escenarios de alto volumen de sensores (QA-3), las cadenas de llamadas síncronas amplifican las latencias. El enfoque basado en eventos elimina este acoplamiento temporal.
Arquitectura de microkernel (plugin-based)	Una arquitectura microkernel facilitaría la modificabilidad (QA-5) mediante plugins, pero su modelo de ejecución centralizado en el núcleo limita la escalabilidad horizontal independiente por componente (QA-3). Asimismo, el mecanismo de plugin introduce latencia adicional en el path de procesamiento de imágenes, comprometiendo QA-2.

Paso 5 — Instanciar Elementos, Establecer Responsabilidades y Definir Interfaces

La descomposición interna del sistema SmartRoad genera los siguientes contenedores con sus responsabilidades y tecnologías asignadas:

Elemento (Contenedor)	Responsabilidad y Tecnología
Portal Web (SPA)	Interfaz principal de usuario para ingenieros viales y supervisores. Provee visualización geoespacial interactiva (mapa ICP), gestión de alertas y generación de reportes. Tecnología: React + Leaflet/MapLibre. Consume API Gateway REST y WebSocket.
API Gateway	Punto de entrada unificado para todas las peticiones externas. Gestiona autenticación (JWT/OAuth2), autorización RBAC, rate limiting y routing hacia los microservicios internos. Tecnología: AWS API Gateway + AWS Cognito.
Servicio de Ingesta IoT	Recibe y valida datos de sensores IoT via MQTT (AWS IoT Core). Implementa buffer local en agentes de campo para operación offline. Publica eventos validados al tópico Kafka 'sensor-data-raw'. Tecnología: AWS IoT Core + Lambda de validación.
Motor de IA — Detección de Deterioro	Consume imágenes del tópico Kafka 'imagen-vial'. Ejecuta modelos de visión computacional (YOLO / Detectron2) para detectar fisuras, baches y hundimientos. Publica resultados al tópico 'deterioro-detectado'. Tecnología: ECS Fargate + SageMaker Endpoints.
Servicio Geoespacial	Gestiona el almacenamiento y consulta de datos geoespaciales. Calcula el Índice de Condición del Pavimento (ICP) por segmento.

	Provee tiles vectoriales al Portal Web. Tecnología: PostGIS en RDS + GeoServer / MapTiler.
Servicio de Reportes y Alertas	Genera reportes en PDF/GeoJSON para entidades gubernamentales. Envía alertas de deterioro crítico via email, SMS y webhook. Exporta datos en formatos PASER y LTPP (R-5). Tecnología: AWS Lambda + SES + SNS.
Event Bus (Apache Kafka)	Columna vertebral de comunicación asíncrona entre microservicios. Garantiza la durabilidad y el orden de los mensajes. Principales tópicos: 'sensor-data-raw', 'imagen-vial', 'deterioro-detectado', 'alerta-critica'. Tecnología: MSK (Amazon Managed Streaming for Apache Kafka).
Base de Datos Operacional	Almacena datos transaccionales de usuarios, segmentos viales y eventos de deterioro. Tecnología: Amazon RDS PostgreSQL con extensión PostGIS para datos geoespaciales. Alta disponibilidad Multi-AZ (QA-1).
Data Lake / Almacenamiento de Imágenes	Almacena imágenes originales y procesadas, series temporales de sensores y datos históricos de deterioro para análisis predictivo. Tecnología: Amazon S3 + AWS Glue + Amazon Athena.
Capa de Observabilidad	Monitorea la salud de todos los microservicios y el pipeline de datos. Provee alertas de infraestructura y métricas de SLA. Tecnología: Amazon CloudWatch + AWS X-Ray + Grafana.

Paso 6 — Esbozar Vistas y Registrar Decisiones de Diseño

La vista generada en esta iteración es el Diagrama de Contenedores (C4 - Nivel 2) del sistema SmartRoad. La arquitectura muestra los siguientes flujos principales:

Flujo de ingesta IoT: Red de Sensores → AWS IoT Core → Servicio de Ingesta → Kafka ('sensor-data-raw') → Servicio Geoespacial → RDS PostGIS / S3 Data Lake.

Flujo de análisis de imágenes: Portal Web / Drones → API Gateway → S3 Data Lake → Kafka ('imagen-vial') → Motor de IA (SageMaker) → Kafka ('deterioro-detectado') → Servicio Geoespacial.

Flujo de visualización: Portal Web (SPA) → API Gateway → Servicio Geoespacial → Tiles vectoriales + datos ICP.

Flujo de alertas: Kafka ('alerta-critica') → Servicio de Reportes y Alertas → Email/SMS/Webhook + Exportación GeoJSON para SINC.

Decisiones de diseño registradas:

DD-2.1: Se adopta Apache Kafka (MSK) como Event Bus central del sistema. Garantiza persistencia, orden y replay de eventos, resolviendo QA-1 y QA-2.

DD-2.2: El Motor de IA se despliega sobre Amazon ECS Fargate con endpoints de inferencia en SageMaker, permitiendo actualización de modelos sin redeploy del microservicio (QA-5, AE-1).

DD-2.3: El Servicio Geoespacial utiliza PostGIS sobre Amazon RDS Multi-AZ como motor de almacenamiento y consulta espacial, garantizando disponibilidad (QA-1) y compatibilidad con estándares GIS (AE-3).

DD-2.4: La autenticación se centraliza en AWS Cognito con OAuth2/JWT. El API Gateway aplica autorización RBAC para todos los endpoints. Todo tráfico usa TLS 1.3 y el almacenamiento en S3/RDS usa AES-256 (QA-4).

DD-2.5: El autoescalado horizontal de microservicios se gestiona mediante Amazon EKS con Horizontal Pod Autoscaler (HPA) basado en métricas de CPU, memoria y longitud de colas Kafka (QA-3).

Paso 7 — Revisar el Objetivo de la Iteración y los Alcances del Diseño

La siguiente tabla muestra el estado de satisfacción de los insumos de diseño al finalizar la Iteración 2:

Insumo de diseño	Estado	Elemento del concepto de diseño
UC-2	Resuelto	Servicio de Ingesta IoT + AWS IoT Core + Tópico Kafka 'sensor-data-raw'
UC-3	Resuelto	Motor de IA (ECS Fargate + SageMaker) + Tópico Kafka 'imagen-vial'
UC-5	Resuelto	Portal Web (SPA) + Servicio Geoespacial (PostGIS + GeoServer)
QA-1	Resuelto	RDS Multi-AZ + MSK replicación multi-zona + EKS autoescalado + CloudWatch
QA-2	Resuelto	Pipeline asíncrono Kafka — Motor IA desacoplado del path de respuesta al usuario
QA-3	Resuelto	Escalado horizontal independiente por microservicio en EKS + MSK particiones Kafka
QA-4	Resuelto	API Gateway + Cognito (OAuth2/JWT) + RBAC + TLS 1.3 en tránsito + AES-256 en reposo (S3/RDS)
QA-5	Resuelto	Nuevos modelos IA = nuevo consumidor Kafka — sin impacto en otros microservicios
R-1 / R-2	Parcial	Servicio Reportes exporta GeoJSON para SINC/INVIAS — detalle de certificación regulatoria pendiente
R-3	Resuelto	Arquitectura 100 % sobre servicios AWS administrados (MSK, RDS, ECS, SageMaker, S3, API GW)
R-4	Resuelto	Agente IoT con buffer local MQTT + sincronización Kafka al restaurar conectividad
R-5	Parcial	Servicio Reportes implementa exportación PASER/LTPP — validación con estándar pendiente
AE-1 / AE-3 / AE-4	Resuelto	Motor IA sobre SageMaker permite actualización de modelos sin impacto; interfaz web GIS-grade; Servicio Ingesta extensible a nuevos protocolos IoT

Conclusión de la revisión: al finalizar la Iteración 2, la mayoría de los drivers críticos han sido resueltos. Los insumos R-1 y R-5 permanecen en estado parcial y requerirían una Iteración 3 para detallar los procesos de certificación regulatoria y validar la compatibilidad de exportación con los estándares

PASER/LTPP. Sin embargo, el diseño es suficientemente sólido para proceder a la etapa de implementación, dado que los drivers de mayor riesgo técnico (QA-2, QA-3, QA-4, QA-5) han sido completamente resueltos.

5. Conclusiones

Sobre la aplicación del método ADD al sistema SmartRoad: La aplicación del método de Diseño Orientado a Atributos permitió estructurar el proceso de toma de decisiones arquitectónicas de manera rigurosa y trazable. Cada decisión adoptada en las dos iteraciones desarrolladas puede ser rastreada hasta un driver específico —ya sea un atributo de calidad, una restricción o un aspecto externo— lo que garantiza que la arquitectura resultante no es producto de preferencias tecnológicas arbitrarias, sino de un análisis sistemático de los requisitos más relevantes para el sistema.

Sobre el valor del enfoque iterativo: El carácter iterativo del método ADD demostró ser especialmente valioso para el caso SmartRoad. La Iteración 1 permitió tomar la decisión estructural más importante —el estilo arquitectónico EDA sobre microservicios— a un nivel de abstracción alto y sin comprometerse prematuramente con tecnologías específicas. La Iteración 2 pudo entonces traducir esa decisión en contenedores concretos con tecnologías AWS, partiendo de una base arquitectónica ya validada. Esta progresión de lo abstracto a lo concreto es una de las ventajas distintivas del método ADD frente a enfoques de diseño más intuitivos.

Sobre la satisfacción de los atributos de calidad: Los atributos de calidad más críticos del sistema —disponibilidad (QA-1), rendimiento (QA-2), escalabilidad (QA-3), seguridad (QA-4) y modificabilidad (QA-5)— fueron abordados de manera explícita y documentada en el proceso. El uso de Apache Kafka como Event Bus resultó ser la decisión de diseño de mayor impacto transversal, al resolver simultáneamente QA-1, QA-2 y QA-3 mediante desacoplamiento temporal, persistencia de mensajes y escalabilidad horizontal. La decisión de desplegar el Motor de IA sobre SageMaker con consumidores Kafka independientes resolvió QA-5 de forma elegante, permitiendo la actualización de modelos sin impacto en otros componentes.

Sobre las restricciones y aspectos externos: Las restricciones del sistema, especialmente R-4 (conectividad intermitente en zonas rurales) y R-3 (presupuesto cloud), condicionaron significativamente las decisiones de diseño. La adopción del patrón offline-first con sincronización Kafka para los agentes IoT de campo es una respuesta directa a las condiciones reales de operación del sistema en el contexto colombiano. Los aspectos externos, particularmente AE-1 (evolución de modelos IA) y AE-4 (estándares IoT emergentes), fueron incorporados en el diseño a través de mecanismos de extensibilidad que garantizan la adaptabilidad del sistema a futuro.

Sobre las limitaciones del diseño actual y pasos siguientes: El diseño preliminar resultante de las dos iteraciones constituye una arquitectura conceptual sólida, pero no una arquitectura definitiva lista para implementación. Los insumos R-1 (cumplimiento regulatorio IGAC/CONPES) y R-5 (compatibilidad PASER/LTPP) permanecen parcialmente resueltos y requerirían una Iteración 3 con participación de expertos en regulación de datos viales en Colombia. Adicionalmente, sería recomendable complementar el proceso con la aplicación del método ATAM (Architecture Tradeoff Analysis Method) para identificar posibles compensaciones entre los atributos de calidad adoptados, especialmente entre rendimiento y seguridad en el pipeline de procesamiento de imágenes.

Reflexión final: El método ADD evidencia que un buen diseño arquitectónico no comienza con la selección de tecnologías, sino con la comprensión profunda de los drivers del sistema. La calidad de los insumos —especialmente la precisión de los escenarios de atributos de calidad— determina directamente la calidad de las decisiones de diseño. En proyectos de la naturaleza de SmartRoad, donde la infraestructura vial monitoreada tiene implicaciones directas en la seguridad ciudadana y la eficiencia del gasto público, esta rigurosidad metodológica no es opcional: es una responsabilidad profesional del arquitecto de software.

Referencias

Cervantes, H., & Kazman, R. (2016). *Designing Software Architectures: A Practical Approach*. Carnegie Mellon University, Software Engineering Institute.

<https://insights.sei.cmu.edu/library/designing-software-architectures-a-practical-approach/>

Hofmeister, C., Kruchten, P., Nord, R. L., Obbink, H., Ran, A., & America, P. (2007). A general model of software architecture design derived from five industrial approaches. *Journal of Systems and Software*, 80(1), 106–126. <https://doi.org/10.1016/J.JSS.2006.05.024>

Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley Professional.

Brown, S. (2022). The C4 model for visualising software architecture. <https://c4model.com>

Amazon Web Services. (2024). *AWS Well-Architected Framework*.

<https://docs.aws.amazon.com/wellarchitected/latest/framework/>